Ethan Tran

ethantran@quwin.dev | linkedin.com/in/kien-ethan-tran | github.com/quwin

EDUCATION

California State University Sacramento Expected December 2027
Bachelor of Science, Computer Science GPA: 3.47

TECHNICAL SKILLS

Languages: Rust, C++, C, Python, Go, SQL
Databases & Data Systems: PostgreSQL, Apache DataFusion, SQLite, MongoDB, Redis, Qdrant, MySQL, Arrow, Parquet
Cloud & Infrastructure: Amazon Web Services (AWS), Google Cloud Platform (GCP), Linux, Docker, Git, GitHub Actions

WORK EXPERIENCE

Software Engineer Intern July 2026 - Present
Perio Specialist

- Architected a database system with application-level replication across networked workstations to track dental implant inventory.
- Implemented transactional inventory updates, idempotent writes, and optimistic concurrency checks to prevent duplicate stock changes and detect conflicting edits.
- Designed persistent offline queues and automated backups to support outage-tolerant data capture and database recovery.
- Built a pre-processing pipeline that standardized patient radiographs into model-ready features to streamline dataset preparation.
- Developed landmark post-processing to align predictions with tooth boundaries, reducing mean localization error by 4% on annotated dental radiographs.

AI Engineering Extern June 2026 - July 2026
Pfizer-hosted Externship Program

- Built a Python document-processing pipeline using OCR to extract, classify, and route content from 1,000+ page pharmaceutical files.
- Implemented configurable dense and sparse embedding indexes in a Qdrant vector database, supporting flexible query execution.
- Benchmarked extraction, retrieval, and routing across segmentation and indexing changes, improving retrieval accuracy by 14%.
- Deployed typed API endpoints returning source-backed results with validity, confidence, and completeness signals for downstream document-processing workflows.
- Delivered a comprehensive technical report that outlining trade-offs, optimization strategies, and deployment recommendations, enabling stakeholders to make informed decisions on the system rollout.

OPEN SOURCE CONTRIBUTIONS

PostegreSQL - Database Management System

- Discovered a query optimization patch regression causing 36% slower execution by overestimating hash-table memory.
- Reviewed a query planner fix for incorrect NULL handling, finding a 3.93x execution slowdown from blocked SQL-function inlining.

Apache DataFusion - SQL Query Engine

- Fixed DataFusion's UNNEST schema inference to preserve Arrow extension metadata for SedonaDB, affecting an ecosystem with 65M+ cumulative downloads.
- Reviewed and proposed a fix for a query optimization issue causing redundant per-row traversal for reused computations.
- Identified and helped resolve a query execution issue ignoring DELETE predicates and removing unintended rows.
- Improved SQL validation by rejecting invalid user-defined function placeholders at definition time instead of runtime.
- Benchmarked an exponential time query planning regression, driving an optimizer redesign leading to up to 2.78× faster execution.

PROJECTS

Distributed API Gateway + Rate Limiter *Go, Lua, Redis, Nginx, Docker, Prometheus, AWS*

- Developed a Go API gateway combining API-key authentication, reverse-proxy routing, quota headers, and per-principal rate limiting.
- Supported fixed-window, token-bucket, and sliding-window policies through interchangeable in-memory and Redis-backed limiter implementations.
- Replaced client-side read-modify-write sequences with atomic Lua scripts to prevent races in shared limiter state.
- Configured Nginx across three Go gateway instances for local multi-replica validation.
- Validated limiter correctness with deterministic tests covering burst refills, expiration boundaries, cancellation, and Redis state transitions.

GPU-Accelerated Postflop Poker Solver *C++, CUDA, FLTK*

- Developed and deployed a custom No-Limit Hold'em postflop solver using Counterfactual Regret Minimization (CFR) across configurable game trees.
- Engineered flattened memory layouts and GPU kernels to maximize parallel execution, increasing iteration throughput by 7%.
- Applied memory-aware batching to support large workloads on consumer GPUs, reducing memory requirements by up to 60%.
- Profiled compute, transfer, and allocation bottlenecks, restructuring memory access and execution to improve cache locality and reduce synchronization overhead.